



Вариант №2622
Лабораторная работа №5
По дисциплине
Программирование

Выполнил студент группы Р3112:
Киселев Тимофей Васильевич

Преподаватель:
Разинкин Александр Владимирович

1. Текст задания

Реализовать консольное приложение, которое реализует управление коллекцией объектов в интерактивном режиме. В коллекции необходимо хранить объекты класса `Ticket`, описание которого приведено ниже.

Разработанная программа должна удовлетворять следующим требованиям:

- Класс, коллекцией экземпляров которого управляет программа, должен реализовывать сортировку по умолчанию.
- Все требования к полям класса (указанные в виде комментариев) должны быть выполнены.
- Для хранения необходимо использовать коллекцию типа `java.util.Hashtable`
- При запуске приложения коллекция должна автоматически заполняться значениями из файла.
- Имя файла должно передаваться программе с помощью: **переменная окружения**.
- Данные должны храниться в файле в формате `xml`
- Чтение данных из файла необходимо реализовать с помощью класса `java.io.InputStreamReader`
- Запись данных в файл необходимо реализовать с помощью класса `java.io.FileOutputStream`
- Все классы в программе должны быть задокументированы в формате `javadoc`.
- Программа должна корректно работать с неправильными данными (ошибки пользовательского ввода, отсутствие прав доступа к файлу и т.п.).

В интерактивном режиме программа должна поддерживать выполнение следующих команд:

- `help` : вывести справку по доступным командам
- `info` : вывести в стандартный поток вывода информацию о коллекции (тип, дата инициализации, количество элементов и т.д.)
- `show` : вывести в стандартный поток вывода все элементы коллекции в строковом представлении
- `insert null {element}` : добавить новый элемент с заданным ключом
- `update id {element}` : обновить значение элемента коллекции, `id` которого равен заданному
- `remove_key null` : удалить элемент из коллекции по его ключу
- `clear` : очистить коллекцию
- `save` : сохранить коллекцию в файл
- `execute_script file_name` : считать и исполнить скрипт из указанного файла. В скрипте содержатся команды в таком же виде, в котором их вводит пользователь в интерактивном режиме.
- `exit` : завершить программу (без сохранения в файл)
- `replace_if_lower null {element}` : заменить значение по ключу, если новое значение меньше старого
- `remove_greater_key null` : удалить из коллекции все элементы, ключ которых превышает заданный
- `remove_lower_key null` : удалить из коллекции все элементы, ключ которых меньше, чем заданный
- `min_by_type` : вывести любой объект из коллекции, значение поля `type` которого является минимальным
- `filter_greater_than_person person` : вывести элементы, значение поля `person` которых больше заданного
- `print_descending` : вывести элементы коллекции в порядке убывания

Формат ввода команд:

- Все аргументы команды, являющиеся стандартными типами данных (примитивные типы, классы-оболочки, `String`, классы для хранения дат), должны вводиться в той же строке, что и имя команды.
- Все составные типы данных (объекты классов, хранящиеся в коллекции) должны вводиться по одному полю в строку.
- При вводе составных типов данных пользователю должно показываться приглашение к вводу, содержащее имя поля (например, "Введите дату рождения:")
- Если поле является `enum`'ом, то вводится имя одной из его констант (при этом список констант должен быть предварительно выведен).
- При некорректном пользовательском вводе (введена строка, не являющаяся именем константы в `enum`'е; введена строка вместо числа; введенное число не входит в указанные границы и т.п.) должно быть показано сообщение об ошибке и предложено повторить ввод поля.
- Для ввода значений `null` использовать пустую строку.
- Поля с комментарием "Значение этого поля должно генерироваться автоматически" не должны вводиться пользователем вручную при добавлении.

Описание хранимых в коллекции классов:

```
public class Ticket {
    private int id; //Значение поля должно быть больше 0, Значение этого поля должно быть уникальным,
Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.ZonedDateTime creationDate; //Поле не может быть null, Значение этого поля должно
генерироваться автоматически
    private Long price; //Поле не может быть null, Значение поля должно быть больше 0
    private TicketType type; //Поле может быть null
    private Person person; //Поле может быть null
}

public class Coordinates {
    private Float x; //Максимальное значение поля: 882, Поле не может быть null
```

```

        private double y; //Максимальное значение поля: 676
    }
    public class Person {
        private java.util.Date birthday; //Поле может быть null
        private double height; //Значение поля должно быть больше 0
        private int weight; //Значение поля должно быть больше 0
        private String passportID; //Длина строки не должна быть больше 42, Длина строки должна быть не
        меньше 4, Поле не может быть null
    }
    public enum TicketType {
        VIP,
        USUAL,
        BUDGETARY,
        CHEAP;
    }
}

```

Отчёт по работе должен содержать:

1. Текст задания.
2. Диаграмма классов разработанной программы.
3. Исходный код программы.
4. Выводы по работе.

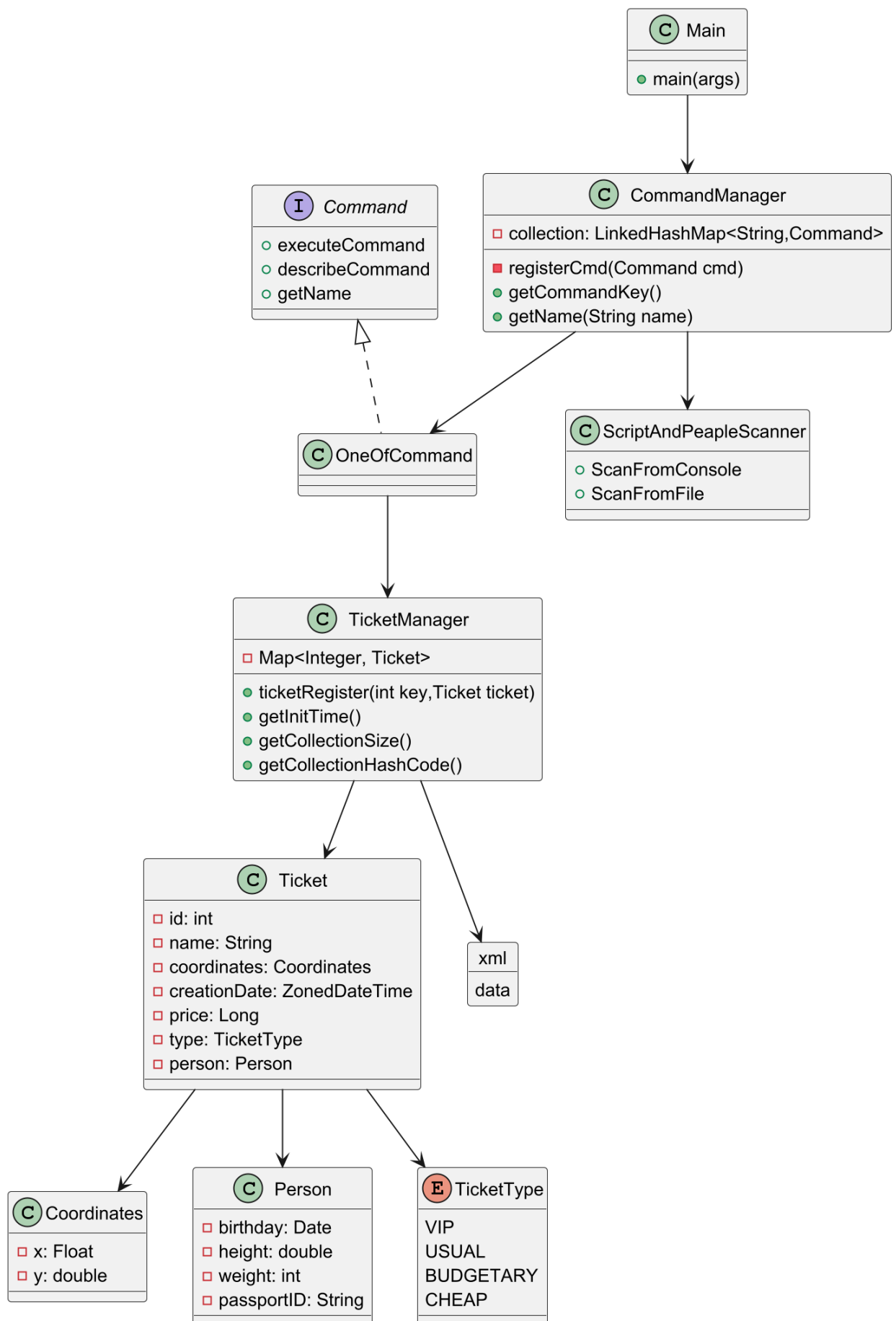
Вопросы к защите лабораторной работы:

1. Коллекции. Сортировка элементов коллекции. Интерфейсы `java.util.Comparable` и `java.util.Comparator`.
2. Категории коллекций - списки, множества, очереди, отображения (словари). Интерфейсы `java.util.List`, `java.util.Set`, `java.util.Queue`, `java.util.Map` и их реализации.
3. Параметризованные типы. Создание параметризуемых классов. Wildcard-параметры.
4. Классы-оболочки. Назначение, область применения, преимущества и недостатки. Автоупаковка и автотораспаковка.
5. Потоки ввода-вывода в Java. Байтовые и символьные потоки. "Цепочки" потоков (Stream Chains).
6. Работа с файлами в Java. Класс `java.io.File`.
7. Пакет `java.nio` - назначение, основные классы и интерфейсы.
8. Утилита `javadoc`. Особенности автоматического документирования кода в Java.

2. Исходный код программы.

<https://github.com/Jikaty/ITMO/tree/master>

3. Диаграмма классов реализованной объектной модели.



4. Вывод

Во время выполнения данной лабораторной работы я научился работать с различными структурами данных в Java и файлами, а также углубил свои знания о ООП в Java, изучил параметризованные типы, wildcard-параметры и утилиту javadoc.